CO3 PRACTICAL EXERCISE

1. Generate text embeddings and perform semantic similarity search.

CODE:

```
import re
from collections import Counter
from math import sqrt

sentences = [
    "Artificial intelligence makes machines intelligent.",
    "Machine learning helps computers learn from data.",
    "Python is a popular programming language.",
    "Generative AI creates new text and images."
]

def embed(text):
    words = re.findall(r"\w+", text.lower())
    return Counter(words)

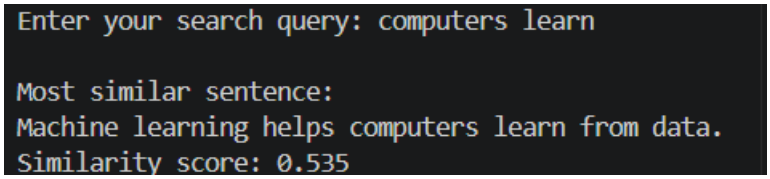
def cosine_similarity(a, b):
    common = set(a) & set(b)
    dot = sum(a[w] * b[w] for w in common)
    norm_a = sqrt(sum(v * v for v in a.values()))
    norm_b = sqrt(sum(v * v for v in b.values()))
    return dot / (norm_a * norm_b) if norm_a and norm_b else 0

query = input("Enter your search query: ")
query_vector = embed(query)

scores = [(s, cosine_similarity(query_vector, embed(s))) for s in sentences]
scores.sort(key=lambda x: x[1], reverse=True)

print("\nMost similar sentence:")
print(scores[0][0])
print("Similarity score:", round(scores[0][1], 3))
```

OUTPUT:



2. Develop a semantic search system using cosine similarity between document and query embeddings.

CODE:

```
import re
from collections import Counter
from math import sqrt

documents = {
    "Document 1": "Artificial intelligence is used in healthcare.",
    "Document 2": "Machine learning is useful for data analysis.",
    "Document 3": "Python is used for software development.",
    "Document 4": "AI helps doctors identify diseases."
}

def vector(text):
    return Counter(re.findall(r"\w+", text.lower()))

def cosine(a, b):
    dot = sum(a[w] * b[w] for w in set(a) & set(b))
    na = sqrt(sum(v*v for v in a.values()))
    nb = sqrt(sum(v*v for v in b.values()))
    return dot / (na * nb) if na and nb else 0

query = input("Enter your search query: ")
query_vector = vector(query)

results = []

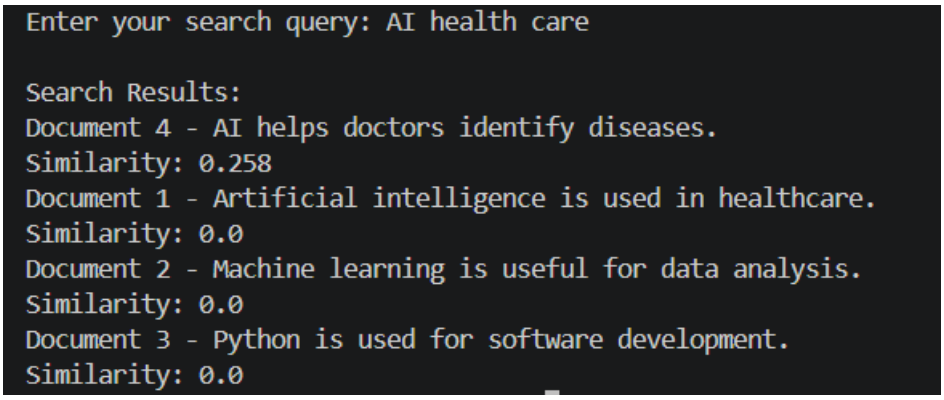
for name, content in documents.items():
```

```
    score = cosine(query_vector, vector(content))
    results.append((name, content, score))

results.sort(key=lambda x: x[2], reverse=True)

print("\nSearch Results:")
for name, content, score in results:
    print(name, "-", content)
    print("Similarity:", round(score, 3))
```

OUTPUT:



3. Build a vector database using FAISS or ChromaDB and perform similarity-based document retrieval.

CODE:

```
import re
from collections import Counter
from math import sqrt

documents = [
    "Python is a programming language.",
    "Artificial intelligence makes machines intelligent.",
    "Machine learning learns patterns from data.",
    "Water conservation helps protect natural resources."
]

def embed(text):
```

```
    return Counter(re.findall(r"\w+", text.lower()))

def similarity(a, b):
    dot = sum(a[w] * b[w] for w in set(a) & set(b))
    na = sqrt(sum(v*v for v in a.values()))
    nb = sqrt(sum(v*v for v in b.values()))
    return dot / (na * nb) if na and nb else 0

vector_database = []

for document in documents:
    vector_database.append({
        "document": document,
        "vector": embed(document)
    })

query = input("Enter your query: ")
query_vector = embed(query)

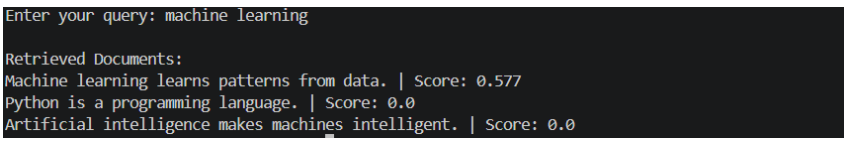
results = []

for item in vector_database:
    score = similarity(query_vector, item["vector"])
    results.append((item["document"], score))

results.sort(key=lambda x: x[1], reverse=True)

print("\nRetrieved Documents:")
for document, score in results[:3]:
    print(document, "| Score:", round(score, 3))
```

OUTPUT:



4. Implement document storage and top-k retrieval using a vector database.

CODE:

```
import re
from collections import Counter
from math import sqrt

documents = [
    "Water is essential for human life.",
    "Smart water systems monitor water flow.",
    "Leak detection helps reduce water loss.",
    "Artificial intelligence supports automation.",
    "Machine learning analyzes sensor data."
]

def embed(text):
    return Counter(re.findall(r"\w+", text.lower()))

def cosine(a, b):
    dot = sum(a[w] * b[w] for w in set(a) & set(b))
    na = sqrt(sum(v*v for v in a.values()))
    nb = sqrt(sum(v*v for v in b.values()))
    return dot / (na * nb) if na and nb else 0

query = input("Enter your query: ")
k = int(input("Enter top-K value: "))

query_vector = embed(query)
results = []

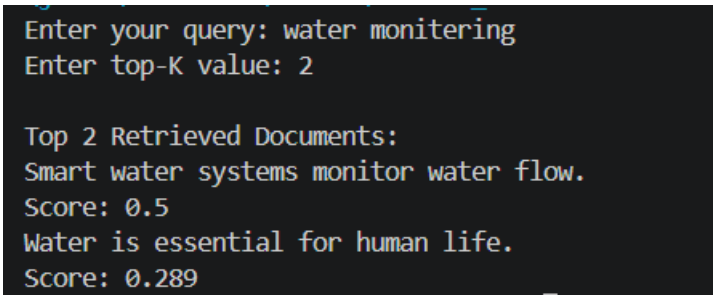
for doc in documents:
    score = cosine(query_vector, embed(doc))
    results.append((doc, score))
```

```
results.sort(key=lambda x: x[1], reverse=True)


print("\nTop", k, "Retrieved Documents:")


for doc, score in results[:k]:
    print(doc)
    print("Score:", round(score, 3))
```

OUTPUT:



5. Develop a document Question Answering system using Retrieval-Augmented Generation (RAG).

CODE:

```
import re
from collections import Counter
from math import sqrt


documents = [
    "Artificial intelligence enables machines to perform intelligent tasks.",
    "Machine learning allows computers to learn from data.",
    "Water conservation reduces unnecessary water usage.",
    "Leak detection identifies damaged water pipelines."
]


def embed(text):
    return Counter(re.findall(r"\w+", text.lower()))


def cosine(a, b):
    dot = sum(a[w] * b[w] for w in set(a) & set(b))
    na = sqrt(sum(v*v for v in a.values()))
```

```
    nb = sqrt(sum(v*v for v in b.values()))
    return dot / (na * nb) if na and nb else 0

question = input("Ask a question: ")
q_vector = embed(question)

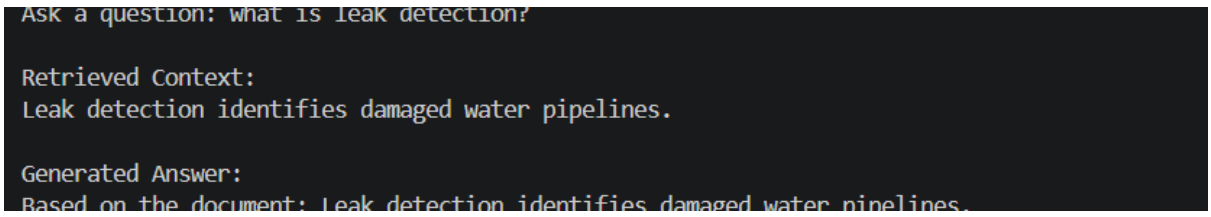
best_doc = max(
    documents,
    key=lambda doc: cosine(q_vector, embed(doc))
)

score = cosine(q_vector, embed(best_doc))

print("\nRetrieved Context:")
print(best_doc)

print("\nGenerated Answer:")
if score > 0:
    print("Based on the document:", best_doc)
else:
    print("No relevant information was found in the documents.")
```

OUTPUT:



6. Build an end-to-end RAG pipeline involving document loading, text chunking,embeddings, retrieval, and answer generation.

CODE:

```
import re
from collections import Counter
from math import sqrt
```

```
documents = [
    "Artificial intelligence is a technology that enables machines to perform tasks. "
    "It is used in healthcare, education, and business.",
    "Machine learning is a part of artificial intelligence. "
    "It learns patterns from data and makes predictions.",
    "Water management includes monitoring water supply. "
    "Leak detection helps prevent water loss."
]

def chunk_text(text):
    sentences = text.split(". ")
    return [s.strip() for s in sentences if s.strip()]

def embed(text):
    return Counter(re.findall(r"\w+", text.lower()))

def cosine(a, b):
    dot = sum(a[w] * b[w] for w in set(a) & set(b))
    na = sqrt(sum(v*v for v in a.values()))
    nb = sqrt(sum(v*v for v in b.values()))
    return dot / (na * nb) if na and nb else 0

chunks = []

for doc in documents:
    chunks.extend(chunk_text(doc))

question = input("Ask your question: ")
q_vector = embed(question)

ranked = sorted(
    chunks,
    key=lambda chunk: cosine(q_vector, embed(chunk)),
```

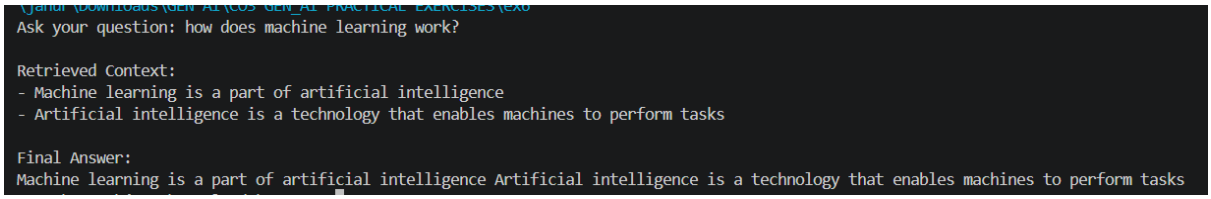
```
    reverse=True
)

context = ranked[:2]

print("\nRetrieved Context:")
for item in context:
    print("-", item)

print("\nFinal Answer:")
if cosine(q_vector, embed(context[0])) > 0:
    print(" ".join(context))
else:
    print("No relevant answer found in the documents.")
```

OUTPUT:



7. Develop a domain-specific chatbot using LangChain and a vector database.

CODE:

```
import re
from collections import Counter
from math import sqrt

knowledge = [
    "Water distribution systems supply water to different areas.",
    "Pipelines transport water from reservoirs to consumers.",
    "Water pressure is important for proper distribution.",
    "Leak detection helps identify damaged pipelines.",
    "Smart sensors monitor flow rate and pressure."
]
```

```
def embed(text):
    return Counter(re.findall(r"\w+", text.lower()))

def cosine(a, b):
    dot = sum(a[w] * b[w] for w in set(a) & set(b))
    na = sqrt(sum(v*v for v in a.values()))
    nb = sqrt(sum(v*v for v in b.values()))
    return dot / (na * nb) if na and nb else 0

print("Water Management Chatbot")
print("Type 'exit' to stop.")

while True:
    question = input("\nYou: ")

    if question.lower() == "exit":
        print("Chatbot: Goodbye!")
        break

    q_vector = embed(question)

    best = max(
        knowledge,
        key=lambda item: cosine(q_vector, embed(item))
    )

    score = cosine(q_vector, embed(best))

    if score > 0:
        print("Chatbot:", best)
    else:
        print("Chatbot: Sorry, I do not have information about that topic.")
```

OUTPUT:

```
Water Management Chatbot
Type 'exit' to stop.

You: What do smart sensors monitor?
Chatbot: Smart sensors monitor flow rate and pressure.
```

8. Implement a context-aware chatbot using LangChain, retrieval, and an LLM.

CODE:

```
import re
from collections import Counter
from math import sqrt

documents = [
    "Python is a programming language.",
    "Machine learning helps computers learn from data.",
    "Artificial intelligence enables machines to perform intelligent tasks.",
    "Generative AI creates text, images, and other content."
]

history = []

def embed(text):
    return Counter(re.findall(r"\w+", text.lower()))

def cosine(a, b):
    dot = sum(a[w] * b[w] for w in set(a) & set(b))
    na = sqrt(sum(v*v for v in a.values()))
    nb = sqrt(sum(v*v for v in b.values()))
    return dot / (na * nb) if na and nb else 0

print("Context-Aware Chatbot")
print("Type 'exit' to stop.")

while True:
    question = input("\nYou: ")
```

```
    if question.lower() == "exit":
        print("Chatbot: Goodbye!")
        break

    context = " ".join(history[-2:])
    combined_query = context + " " + question

    query_vector = embed(combined_query)

    best = max(
        documents,
        key=lambda doc: cosine(query_vector, embed(doc))
    )

    score = cosine(query_vector, embed(best))

    if score > 0:
        answer = best
    else:
        answer = "Sorry, I could not find relevant information."

    print("Chatbot:", answer)

    history.append(question)
```

OUTPUT:

```
Context-Aware Chatbot
Type 'exit' to stop.

You:  What is machine learning?
Chatbot: Machine learning helps computers learn from data.

You: What does it help computers do?
Chatbot: Machine learning helps computers learn from data.
```

9. Develop a simple AI assistant capable of answering questions using external documents.

CODE:

```
import re
from collections import Counter
from math import sqrt

document = """
Artificial intelligence enables machines to perform intelligent tasks.
Python is a popular programming language.
Machine learning helps computers learn from data.
Generative AI creates new content such as text and images.
"""

def embed(text):
    return Counter(re.findall(r"\w+", text.lower()))

def cosine(a, b):
    dot = sum(a[w] * b[w] for w in set(a) & set(b))
    na = sqrt(sum(v*v for v in a.values()))
    nb = sqrt(sum(v*v for v in b.values()))
    return dot / (na * nb) if na and nb else 0

sentences = [
    s.strip() for s in document.split(".") if s.strip()
]

print("Document AI Assistant")
print("Type 'exit' to stop.")

while True:
    question = input("\nAsk a question: ")

    if question.lower() == "exit":
        print("Assistant: Thank you!")
        break
```

```
    best = max(
        sentences,
        key=lambda s: cosine(embed(question), embed(s))
    )

    score = cosine(embed(question), embed(best))

    if score > 0:
        print("Assistant:", best)
    else:
        print("Assistant: No matching information found.")
```

OUTPUT:

```
Document AI Assistant
Type 'exit' to stop.

Ask a question: what is gen ai?
Assistant: Python is a popular programming language

Ask a question: what is GEN AI?
Assistant: Python is a popular programming language
```

10. Build a document-based AI assistant that supports multiple uploaded documents and question answering.

CODE:

```
import os
import re
from collections import Counter
from math import sqrt

folder = "documents"

os.makedirs(folder, exist_ok=True)

sample_files = {
    "ai.txt": "Artificial intelligence enables machines to perform intelligent tasks.",
    "python.txt": "Python is a popular programming language used for development.",
```

```
    "ml.txt": "Machine learning allows computers to learn patterns from data.",
    "water.txt": "Smart water systems monitor flow rate, pressure, and water leakage."
}

for filename, content in sample_files.items():
    path = os.path.join(folder, filename)

    if not os.path.exists(path):
        with open(path, "w", encoding="utf-8") as file:
            file.write(content)

def embed(text):
    return Counter(re.findall(r"\w+", text.lower()))

def cosine(a, b):
    dot = sum(a[w] * b[w] for w in set(a) & set(b))
    na = sqrt(sum(v*v for v in a.values()))
    nb = sqrt(sum(v*v for v in b.values()))
    return dot / (na * nb) if na and nb else 0

documents = []

for filename in os.listdir(folder):
    if filename.endswith(".txt"):
        path = os.path.join(folder, filename)

        with open(path, "r", encoding="utf-8") as file:
            content = file.read()

        documents.append((filename, content))

print("Multiple Document AI Assistant")
print("Loaded documents:", len(documents))
print("Type 'exit' to stop.")
```

```
while True:
    question = input("\nAsk a question: ")

    if question.lower() == "exit":
        print("Assistant: Goodbye!")
        break

    results = []

    for filename, content in documents:
        sentences = [
            s.strip() for s in content.split(".") if s.strip()
        ]

        for sentence in sentences:
            score = cosine(embed(question), embed(sentence))
            results.append((filename, sentence, score))

    results.sort(key=lambda x: x[2], reverse=True)

    if results and results[0][2] > 0:
        filename, answer, score = results[0]

        print("\nAssistant:", answer)
        print("Source Document:", filename)
        print("Similarity Score:", round(score, 3))
    else:
 print("Assistant: No relevant information found.")
```

OUTPUT:

